

FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Campo	Información
Departamento	Departamento de Ciencias de la Computación
Carrera	Ingeniería en Software
Asignatura	Análisis y Diseño de Software
NRC	30723
Integrantes	Viscaino Jordy, Lucas Gongora, Klever Jami
Proyecto	PaperBoost
Documento	Construcción con la Implementación de patrones de diseño y arquitectura

CONSTRUCCIÓN DE PAPERBOOST CON ARQUITECTURA DE TRES CAPAS Y PATRÓN SINGLETON
REQUISITOS FUNCIONALES DE NIVEL 0

ANTECEDENTES

1. PaperBoost es una aplicación móvil destinada a la administración sencilla de inventario y ventas en pequeños negocios. El usuario principal es el administrador del establecimiento, por lo que el sistema prioriza formularios breves, mensajes comprensibles y operaciones directas.
2. La arquitectura base del proyecto se organiza en tres capas: Presentación, Lógica de Negocio y Datos. Esta estructura separa la interacción con el usuario, las reglas operativas y el almacenamiento de la información.
3. El análisis previo de patrones seleccionó Singleton como patrón principal y limitó su aplicación a SessionManager. El patrón mantiene una sola sesión activa del administrador y se integra dentro de la capa de Lógica de Negocio, sin reemplazar las tres capas ni crear una capa adicional.
4. Los requisitos funcionales de nivel 0 de PaperBoost abarcan el núcleo de gestión de productos y el control de acceso. El CRUD priorizado está conformado por RQF-007 Registrar producto, RQF-011 Consultar inventario, RQF-013 Editar producto y RQF-023 Dar de baja producto, junto con RQF-008, RQF-012, RQF-014 y RQF-024 como requisitos asociados. El inicio de sesión se incorpora mediante RQF-001 Validar credenciales y RQF-002 Proteger las credenciales ingresadas.
5. La implementación inicial utiliza almacenamiento administrado por la aplicación. Los repositorios abstraen la forma de guardar y recuperar datos, permitiendo que la Presentación y la Lógica de Negocio se mantengan independientes del mecanismo concreto de almacenamiento.

OBJETIVO GENERAL

Definir la estructura técnica de PaperBoost para implementar los requisitos funcionales de nivel 0, incluyendo el CRUD priorizado de productos y el inicio de sesión, mediante una arquitectura de tres capas e integrando el patrón Singleton en SessionManager, de forma que los modelos PlantUML sirvan como guía directa para la construcción del sistema.

OBJETIVOS ESPECÍFICOS

- Delimitar las responsabilidades de Presentación, Lógica de Negocio y Datos.
- Modelar los componentes necesarios para validar el acceso, registrar, consultar, editar y dar de baja productos.
- Ubicar SessionManager como único Singleton dentro de la capa de Lógica de Negocio.
- Definir relaciones permitidas y prohibidas entre capas.
- Proporcionar código PlantUML de arquitectura, componentes, clases y secuencias.
- Establecer una matriz de trazabilidad entre requisitos, clases, métodos, validaciones y criterios de aceptación.
- Convertir los modelos en una guía directa para la construcción del sistema durante los sprints.

DESARROLLO

PARTE 1. DELIMITACIÓN DEL PROBLEMA Y REQUISITOS FUNCIONALES DE NIVEL 0

1.1 Descripción del problema

El núcleo funcional de PaperBoost debe permitir que el administrador acceda mediante credenciales y, una vez autenticado, registre productos, consulte el inventario, modifique información existente y retire productos que ya no se encuentren disponibles. Estas operaciones no deben concentrarse en las pantallas,

porque la interfaz terminaría validando acceso, administrando colecciones y modificando estados directamente.

La solución consiste en conservar una comunicación secuencial: la Presentación captura la acción, un controlador coordina el caso de uso, un servicio aplica las reglas y un repositorio administra los registros. La entidad Product representa los datos del dominio. SessionManager se utiliza únicamente para conservar y consultar la sesión activa.

1.2 Requisitos funcionales de nivel 0

ID	Requisito	Operación	Prioridad	Caso de uso
RQF-007	El programa deberá permitir registrar productos manualmente mediante un formulario.	Crear	Alta	CUC-04
RQF-011	El programa deberá mostrar una lista de los productos registrados en el inventario.	Consultar	Alta	CUC-06
RQF-013	El programa deberá permitir modificar la información de un producto registrado.	Actualizar	Alta	CUC-07
RQF-023	El programa deberá permitir dar de baja un producto registrado.	Baja lógica	Alta	CUC-12
RQF-001	El programa deberá permitir al administrador iniciar sesión mediante correo electrónico y contraseña.	Autenticar	Alta	CUC-01
RQF-002	El programa deberá proteger las credenciales ingresadas por el administrador durante el proceso de inicio de sesión.	Proteger credenciales	Alta	CUC-01

1.3 Requisitos asociados

ID	Relación con el CRUD	Aporte
RQF-008	Registro	Valida campos obligatorios y evita SKU repetidos.
RQF-012	Consulta	Permite buscar, filtrar y ordenar productos.
RQF-014	Edición	Permite descartar cambios antes de guardar.
RQF-024	Baja	Solicita confirmación y mantiene el producto activo al cancelar.
RQF-001 / RQF-002	Acceso	Permiten activar una sesión única y proteger las operaciones del administrador.
RQF-002	Inicio de sesión	Oculto la contraseña, evita su exposición y limpia los campos después de intentos incorrectos o cierre de sesión.

PARTE 2. ARQUITECTURA BASE DE TRES CAPAS

2.1 Responsabilidades de las capas

Capa	Componentes principales	Responsabilidad
Presentación	LoginView, ProductFormView, InventoryView, ProductEditView, ProductDetailView, ConfirmDeactivateDialog	Capturar datos, ejecutar acciones de interfaz y mostrar resultados. No valida reglas de negocio ni accede directamente a repositorios.
Lógica de Negocio	AuthController, ProductController, AuthService, ProductService, ProductValidator, OperationResult, SessionManager	Coordinar casos de uso, aplicar validaciones, controlar sesión, construir respuestas y decidir cuándo consultar o modificar datos.
Datos	UserRepository, ProductRepository, User, Product, Category	Guardar, buscar, actualizar, listar y cambiar el estado de los registros administrados por la aplicación.

2.2 Reglas de comunicación

- Las vistas se comunican únicamente con controladores.
- Los controladores delegan las reglas a servicios.
- Los servicios consultan repositorios y validadores.
- Los repositorios administran entidades y colecciones.
- Las vistas pueden mostrar entidades o DTO de respuesta, pero no deben modificarlas directamente.
- SessionManager puede ser consultado por servicios protegidos, pero no almacena productos, ventas ni alertas.
- Los repositorios no conocen las vistas ni los controladores.

Modelo arquitectónico general en PlantUML

```
@startuml
skinparam shadowing false
skinparam componentStyle rectangle
skinparam packageStyle rectangle
skinparam defaultTextAlignment center
left to right direction

title PaperBoost - Arquitectura de Tres Capas con Singleton

skinparam package {
    BackgroundColor<<presentacion>> #D6EAF8
    BorderColor<<presentacion>> #2980B9
    BackgroundColor<<logica>> #D5F5E3
    BorderColor<<logica>> #27AE60
    BackgroundColor<<datos>> #FCE4D6
    BorderColor<<datos>> #C55A11
}

actor "Administrador" as Admin

package "Capa de Presentación" <<presentacion>> {
    component "LoginView" as LoginView
    component "ProductFormView" as ProductForm
    component "InventoryView" as Inventory
    component "ProductEditView" as EditView
    component "ConfirmDeactivateDialog" as ConfirmDialog
}

package "Capa de Lógica de Negocio" <<logica>> {
    component "AuthController" as AuthController
    component "ProductController" as ProductController
    component "AuthService" as AuthService
    component "ProductService" as ProductService
}
```

```

component "ProductValidator" as Validator
component "OperationResult" as Result
component "SessionManager\n<<Singleton>>" as Session
}

package "Capa de Datos" <<datos>> {
  component "UserRepository" as UserRepo
  component "ProductRepository" as ProductRepo
  component "User" as User
  component "Product" as Product
  component "Category" as Category
}

Admin --> LoginView
Admin --> ProductForm
Admin --> Inventory
Admin --> EditView
Admin --> ConfirmDialog

LoginView --> AuthController : solicita inicio/cierre
AuthController --> AuthService
AuthService --> UserRepo : valida credenciales
AuthService --> Session : activa / cierra sesión
AuthController --> Result

ProductForm --> ProductController : registrar
Inventory --> ProductController : listar / buscar
EditView --> ProductController : actualizar
ConfirmDialog --> ProductController : confirmar baja

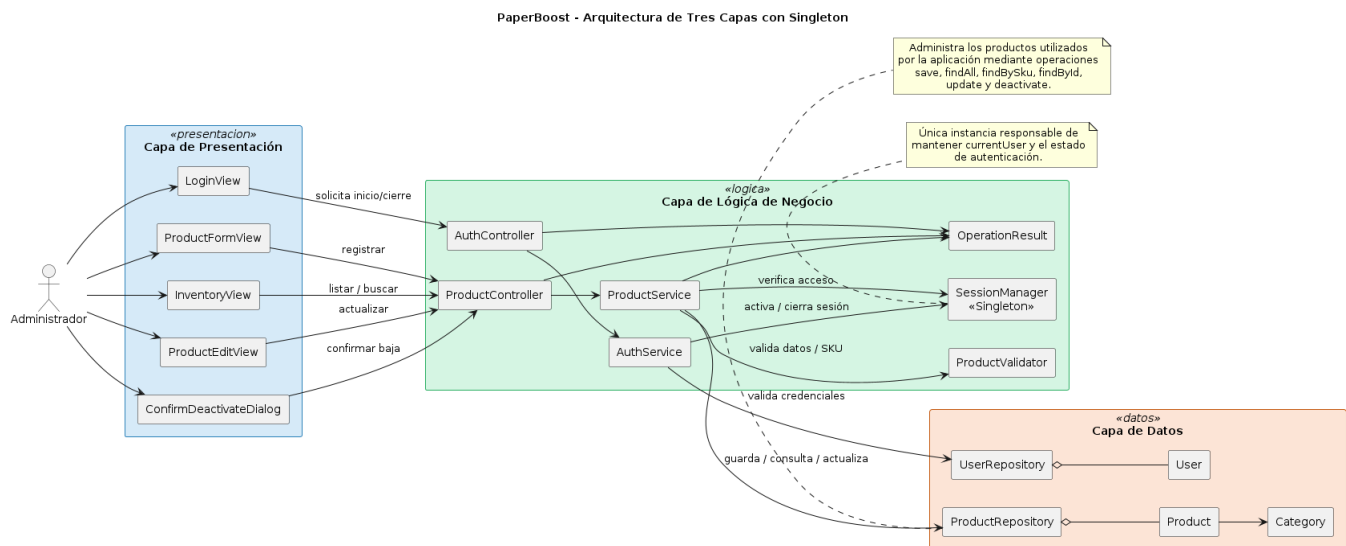
ProductController --> ProductService
ProductService --> Session : verifica acceso
ProductService --> Validator : valida datos / SKU
ProductService --> ProductRepo : guarda / consulta / actualiza
ProductService --> Result
ProductController --> Result

UserRepo o-- User
ProductRepo o-- Product
Product --> Category

note bottom of Session
Única instancia responsable de
mantener currentUser y el estado
de autenticación.
end note

note bottom of ProductRepo
Administra los productos utilizados
por la aplicación mediante operaciones
save, findAll, findBySku, findById,
update y deactivate.
end note
@enduml

```



2.3 Interpretación del modelo

El diagrama coloca explícitamente todos los componentes dentro de una de las tres capas. Singleton aparece como estereotipo de SessionManager, pero continúa contenido en Lógica de Negocio. La dirección principal de dependencias es Presentación hacia Lógica y Lógica hacia Datos. No se representa ninguna conexión directa entre vistas y repositorios.

PARTE 3. INTEGRACIÓN DEL PATRÓN SINGLETON

3.1 Problema resuelto por el patrón

Las distintas operaciones protegidas necesitan consultar el mismo estado de autenticación. Si cada controlador o servicio mantuviera su propia copia del usuario actual, podrían existir estados contradictorios. SessionManager centraliza esta información en una única instancia.

3.2 Responsabilidad limitada

SessionManager debe	SessionManager no debe
Mantener currentUser.	Guardar productos.
Indicar si existe una sesión activa.	Validar SKU.
Activar la sesión después de validar credenciales.	Calcular ventas.
Cerrar la sesión.	Acceder a las pantallas.
Entregar el usuario actual a servicios autorizados.	Convertir todos los servicios o repositorios en Singleton.

Definición estática de SessionManager

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

title PaperBoost - Definición del patrón Singleton

class SessionManager <<Singleton>> {
    - {static} instance : SessionManager
}
```

```

- currentUser : User
- SessionManager()
+ {static} getInstance() : SessionManager
+ login(user : User) : void
+ logout() : void
+ isAuthenticated() : boolean
+ getCurrentUser() : User
}

```

```

class User {
- id : String
- email : String
- password : String
- role : String
}

```

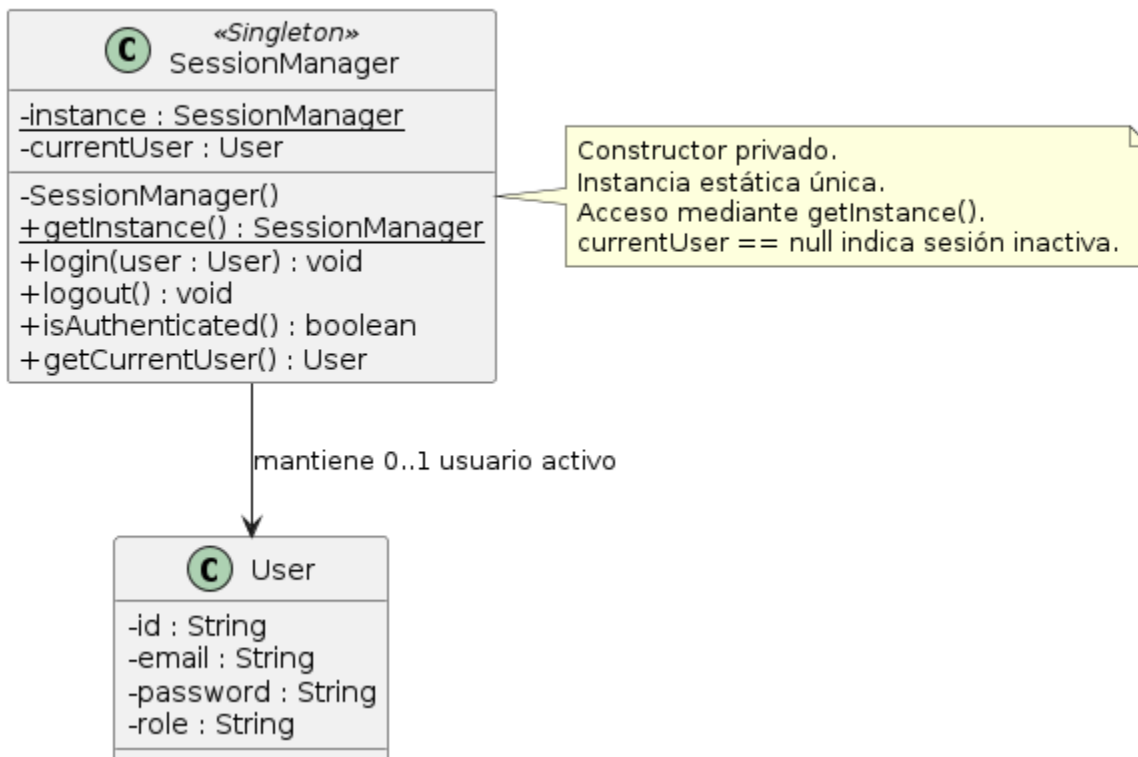
SessionManager --> User : mantiene 0..1 usuario activo

```

note right of SessionManager
Constructor privado.
Instancia estática única.
Acceso mediante getInstance().
currentUser == null indica sesión inactiva.
end note
@enduml

```

PaperBoost - Definición del patrón Singleton



Secuencia de activación de sesión

```

@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center

```

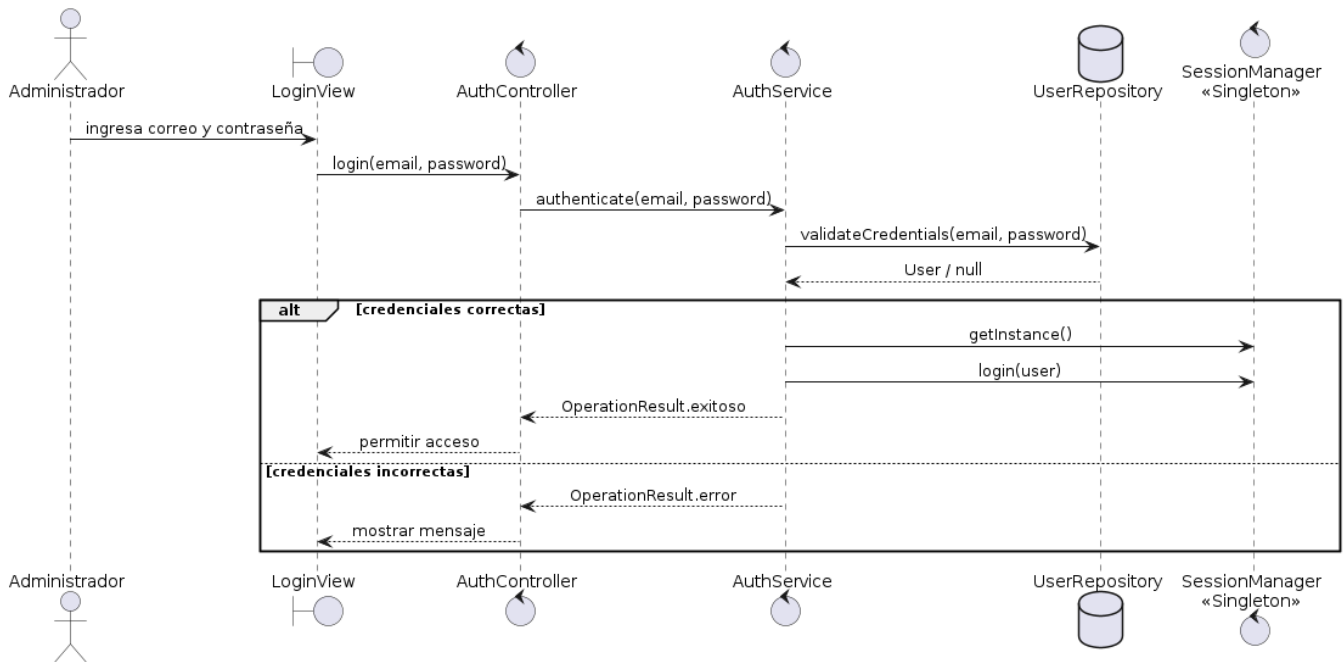
title Flujo de autenticación y activación de SessionManager

actor Administrador
 boundary LoginView
 control AuthController
 control AuthService
 database UserRepository
 control "SessionManager\n<<Singleton>>" as Session

Administrador -> LoginView : ingresa correo y contraseña
 LoginView -> AuthController : login(email, password)
 AuthController -> AuthService : authenticate(email, password)
 AuthService -> UserRepository : validateCredentials(email, password)
 UserRepository --> AuthService : User / null

alt credenciales correctas
 AuthService -> Session : getInstance()
 AuthService -> Session : login(user)
 AuthService --> AuthController : OperationResult.exitoso
 AuthController --> LoginView : permitir acceso
else credenciales incorrectas
 AuthService --> AuthController : OperationResult.error
 AuthController --> LoginView : mostrar mensaje
end
@enduml

Flujo de autenticación y activación de SessionManager



PARTE 4. DISEÑO DE LOS REQUISITOS FUNCIONALES DE NIVEL 0

4.1 Componentes y responsabilidades

Componente	Capa	Responsabilidad
ProductFormView	Presentación	Recibir los datos de un producto nuevo y mostrar el resultado del registro.
InventoryView	Presentación	Mostrar productos, resultados de búsqueda, filtros y estados de stock.

Componente	Capa	Responsabilidad
ProductEditView	Presentación	Cargar datos actuales, permitir modificaciones y cancelar cambios.
ConfirmDeactivateDialog	Presentación	Solicitar confirmación antes de la baja lógica.
ProductController	Lógica	Recibir solicitudes de las vistas y devolver <code>OperationResult</code> .
ProductService	Lógica	Aplicar las reglas de registrar, listar, buscar, actualizar y dar de baja.
ProductValidator	Lógica	Validar campos, valores numéricos y datos obligatorios.
SessionManager	Lógica	Mantener una única sesión activa y validar acceso.
ProductRepository	Datos	Administrar la colección de productos.
Product	Datos / entidad	Representar SKU, código, nombre, precio, stock, categoría, ubicación y estado.
OperationResult	Lógica / DTO	Encapsular éxito, mensaje y datos devueltos a la interfaz.
LoginView	Presentación	Capturar correo y contraseña, solicitar el inicio o cierre de sesión y mostrar mensajes de acceso.
AuthController	Lógica	Recibir solicitudes de <code>LoginView</code> y delegar la validación de credenciales.
AuthService	Lógica	Validar campos, consultar <code>UserRepository</code> y activar o cerrar la sesión.
UserRepository	Datos	Administrar los usuarios registrados y comparar las credenciales ingresadas.
User	Datos / entidad	Representar identificador, correo, contraseña y rol del administrador.

Diagrama de clases estático

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam packageStyle rectangle
left to right direction

title PaperBoost - Diagrama de clases de requisitos nivel 0

package "Presentación" #D6EAF8 {
    class LoginView {
        - authController : AuthController
        + getEmail() : String
        + getPassword() : String
        + displayResult(result : OperationResult)
    }
    class ProductFormView {
        - productController : ProductController
    }
}

```

```

    + getProductData() : ProductDTO
    + displayResult(result : OperationResult)
}
class InventoryView {
    - productController : ProductController
    + showProducts(products : List<Product>)
    + getSearchCriteria() : SearchCriteria
}
class ProductEditView {
    - productController : ProductController
    - originalProduct : Product
    + loadProduct(product : Product)
    + getEditedData() : ProductDTO
    + cancelEdition()
}
class ConfirmDeactivateDialog {
    + confirm(productId : String)
    + cancel()
}
}

package "Lógica de Negocio" #D5F5E3 {
    class AuthController {
        - authService : AuthService
        + login(email : String, password : String) : OperationResult
        + logout() : OperationResult
    }
    class AuthService {
        - userRepository : UserRepository
        - sessionManager : SessionManager
        + authenticate(email : String, password : String) : OperationResult
        + closeSession() : OperationResult
    }
    class ProductController {
        - productService : ProductService
        + createProduct(data : ProductDTO) : OperationResult
        + listProducts() : OperationResult
        + searchProducts(criteria : SearchCriteria) : OperationResult
        + updateProduct(id : String, data : ProductDTO) : OperationResult
        + deactivateProduct(id : String) : OperationResult
    }
    class ProductService {
        - repository : ProductRepository
        - validator : ProductValidator
        - sessionManager : SessionManager
        + registerProduct(data : ProductDTO) : OperationResult
        + getActiveProducts() : OperationResult
        + searchProducts(criteria : SearchCriteria) : OperationResult
        + updateProduct(id : String, data : ProductDTO) : OperationResult
        + deactivateProduct(id : String) : OperationResult
    }
    class ProductValidator {
        + validateRequiredFields(data : ProductDTO) : OperationResult
        + validateNumericValues(data : ProductDTO) : OperationResult
        + validateSkuUnique(sku : String, currentId : String) : OperationResult
    }
    class OperationResult {
        - success : boolean
        - message : String
        - data : Object
        + ok(message : String, data : Object) : OperationResult
        + error(message : String) : OperationResult
    }
}

```

```

    }
    class SessionManager <<Singleton>> {
        - {static} instance : SessionManager
        - currentUser : User
        - SessionManager()
        + {static} getInstance() : SessionManager
        + login(user : User)
        + logout()
        + isAuthenticated() : boolean
        + getCurrentUser() : User
    }
}

package "Datos" #FCE4D6 {
    class UserRepository {
        - users : List<User>
        + findByEmail(email : String) : User
        + validateCredentials(email : String, password : String) : User
    }
    class ProductRepository {
        - products : List<Product>
        + save(product : Product) : Product
        + findAllActive() : List<Product>
        + findById(id : String) : Product
        + findBySku(sku : String) : Product
        + search(criteria : SearchCriteria) : List<Product>
        + update(product : Product) : Product
        + deactivate(id : String) : boolean
    }
    class User {
        - id : String
        - email : String
        - password : String
        - role : String
    }
    class Product {
        - id : String
        - sku : String
        - barcode : String
        - name : String
        - price : Decimal
        - stock : int
        - category : String
        - location : String
        - status : ProductStatus
    }
}

LoginView --> AuthController
AuthController --> AuthService
AuthController --> OperationResult
AuthService --> UserRepository
AuthService --> SessionManager
UserRepository o-- "0..*" User
SessionManager --> User : currentUser

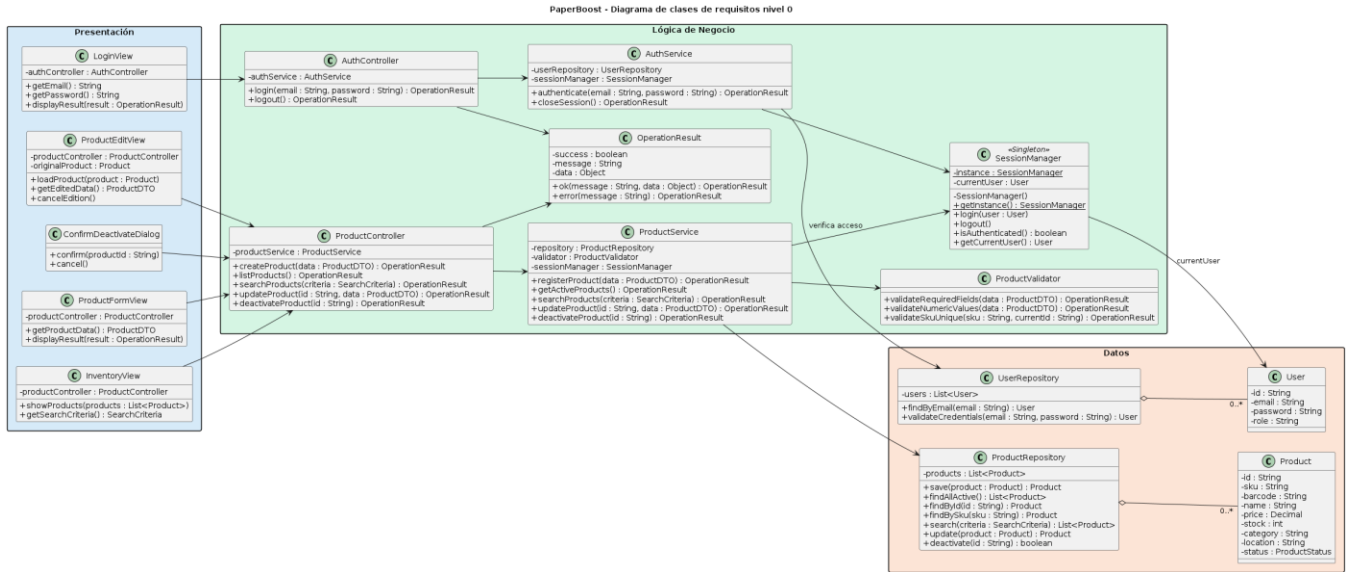
ProductFormView --> ProductController
InventoryView --> ProductController
ProductEditView --> ProductController
ConfirmDeactivateDialog --> ProductController
ProductController --> ProductService
ProductController --> OperationResult

```

```

ProductService --> ProductValidator
ProductService --> ProductRepository
ProductService --> SessionManager : verifica acceso
ProductRepository o-- "0..*" Product
@enduml

```



4.2 RQF-001 y RQF-002 - Inicio de sesión y protección de credenciales

El administrador ingresa correo electrónico y contraseña. AuthService valida los campos, consulta las credenciales administradas por UserRepository y, cuando son correctas, registra al usuario en la única instancia de SessionManager. La contraseña permanece oculta en la interfaz y no se incluye en mensajes de respuesta.

Responsabilidades de implementación

- Implementar LoginView con campos para correo electrónico y contraseña.
- Configurar el campo de contraseña para ocultar los caracteres ingresados.
- Validar que ambos campos se encuentren completos.
- Comparar los datos con las credenciales registradas mediante UserRepository.
- Activar SessionManager cuando las credenciales sean correctas.
- Limpiar los campos y mostrar un mensaje cuando el acceso sea rechazado.
- Permitir el cierre de sesión mediante SessionManager.logout().

Diagrama de secuencia PlantUML - RQF-001 y RQF-002

```

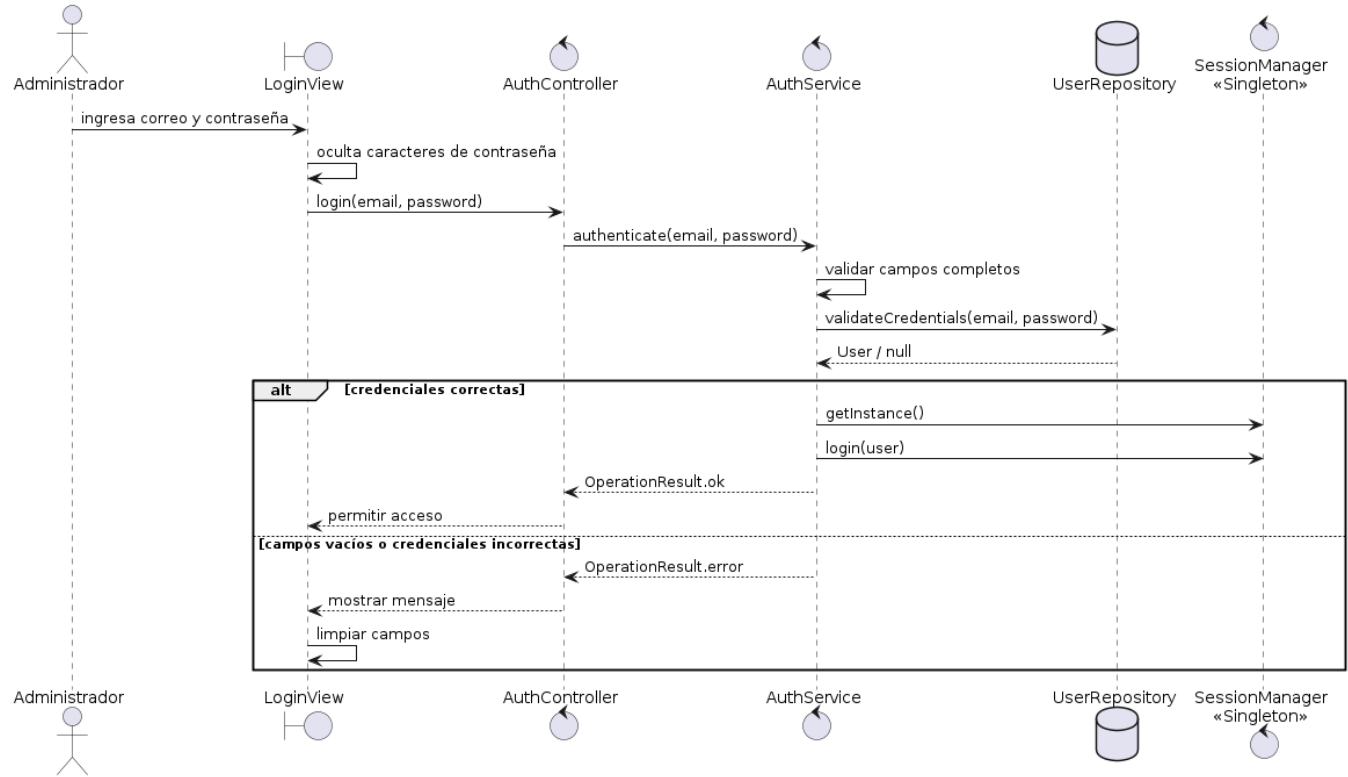
@startuml
actor Administrador
boundary LoginView
control AuthController
control AuthService
database UserRepository
control "SessionManager\n<<Singleton>>" as Session

Administrador -> LoginView : ingresa correo y contraseña
LoginView -> LoginView : oculta caracteres de contraseña
LoginView -> AuthController : login(email, password)
AuthController -> AuthService : authenticate(email, password)
AuthService -> AuthService : validar campos completos
AuthService -> UserRepository : validateCredentials(email, password)
UserRepository --> AuthService : User / null

```

```

alt credenciales correctas
  AuthService -> Session : getInstance()
  AuthService -> Session : login(user)
  AuthService --> AuthController : OperationResult.ok
  AuthController --> LoginView : permitir acceso
else campos vacíos o credenciales incorrectas
  AuthService --> AuthController : OperationResult.error
  AuthController --> LoginView : mostrar mensaje
  LoginView -> LoginView : limpiar campos
end
@enduml
  
```



4.3 RQF-007 - Registro manual de productos

El administrador completa un formulario y el sistema registra un producto válido. ProductService coordina la validación y el almacenamiento.

Responsabilidades de implementación

- Capturar nombre, SKU, precio, cantidad, categoría y ubicación.
- Validar campos obligatorios y valores numéricos.
- Comprobar que el SKU no exista.
- Crear la entidad Product con estado ACTIVE.
- Guardar mediante ProductRepository.
- Devolver un mensaje y actualizar el inventario.

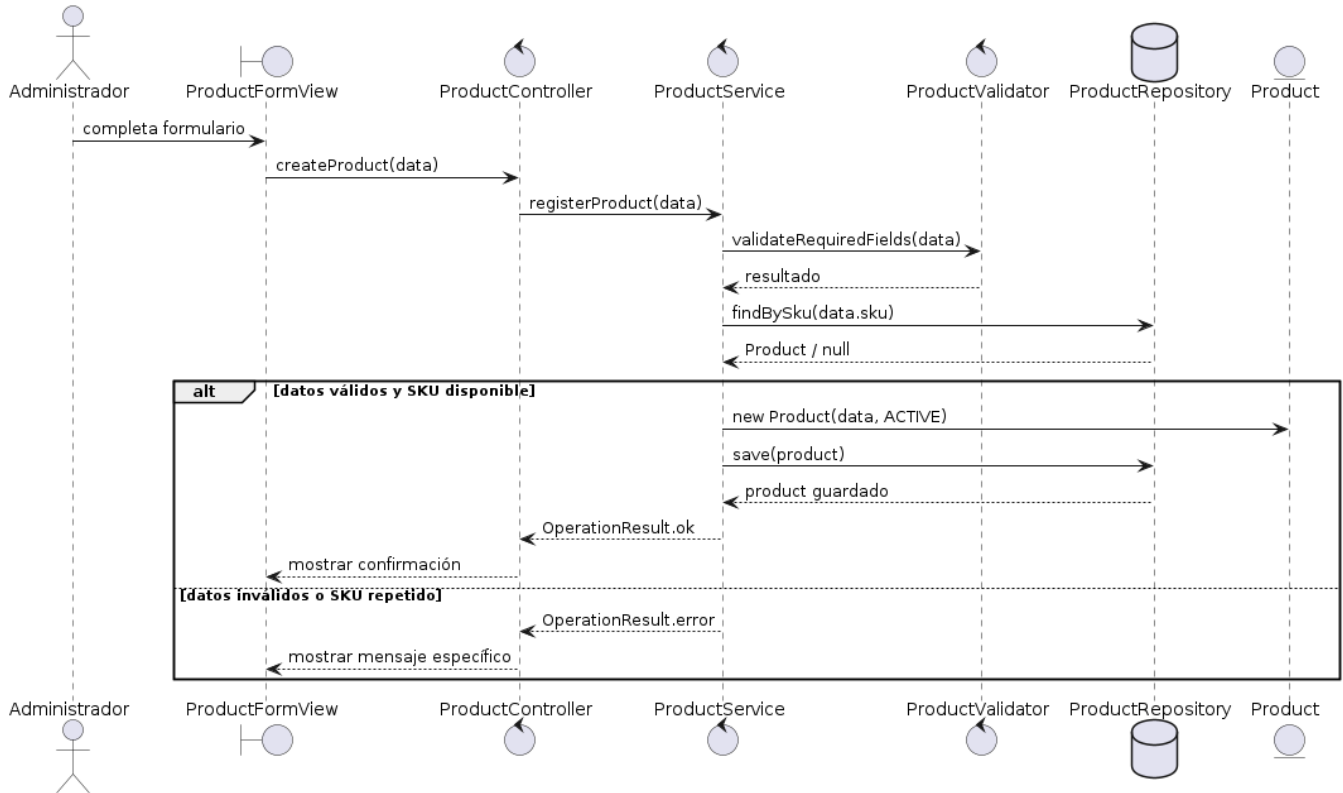
Diagrama de secuencia PlantUML - RQF-007

```

@startuml
actor Administrador
  
```

```
boundary ProductFormView
control ProductController
control ProductService
control ProductValidator
database ProductRepository
entity Product
```

```
Administrador -> ProductFormView : completa formulario
ProductFormView -> ProductController : createProduct(data)
ProductController -> ProductService : registerProduct(data)
ProductService -> ProductValidator : validateRequiredFields(data)
ProductValidator --> ProductService : resultado
ProductService -> ProductRepository : findBySku(data.sku)
ProductRepository --> ProductService : Product / null
alt datos válidos y SKU disponible
  ProductService -> Product : new Product(data, ACTIVE)
  ProductService -> ProductRepository : save(product)
  ProductRepository --> ProductService : product guardado
  ProductService --> ProductController : OperationResult.ok
  ProductController --> ProductFormView : mostrar confirmación
else datos inválidos o SKU repetido
  ProductService --> ProductController : OperationResult.error
  ProductController --> ProductFormView : mostrar mensaje específico
end
@endum1
```



4.4 RQF-011 - Consulta del inventario

La vista solicita los productos activos y presenta los campos principales. El repositorio conserva la responsabilidad de recuperar los registros.

Responsabilidades de implementación

- Solicitar la lista desde InventoryView.
- Verificar que exista una sesión activa.
- Consultar productos con estado ACTIVE.
- Calcular o asignar el estado visual de stock.

- Devolver la lista ordenada para presentación.
- Mostrar un mensaje cuando no existan productos.

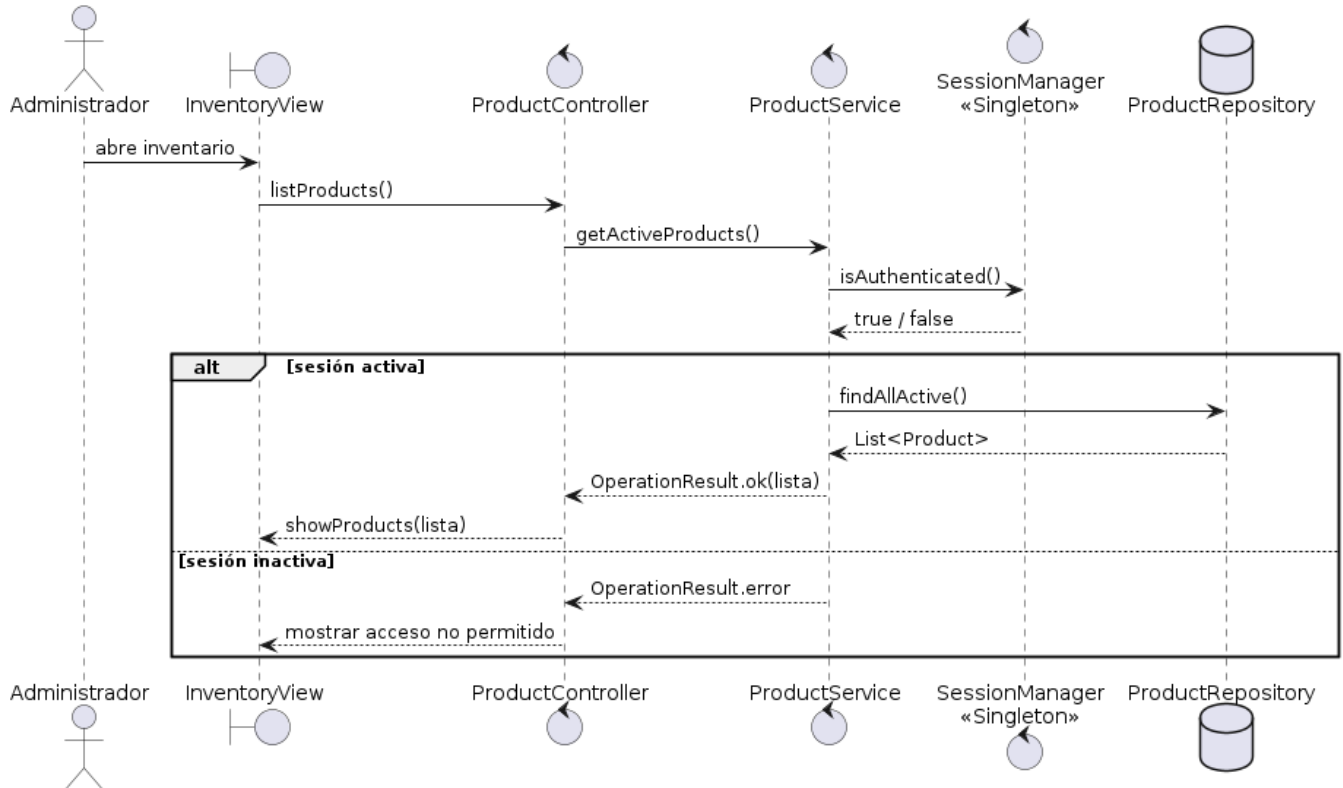
Diagrama de secuencia PlantUML - RQF-011

```

@startuml
actor Administrador
boundary InventoryView
control ProductController
control ProductService
control "SessionManager\n<<Singleton>>" as Session
database ProductRepository

Administrador -> InventoryView : abre inventario
InventoryView -> ProductController : listProducts()
ProductController -> ProductService : getActiveProducts()
ProductService -> Session : isAuthenticated()
Session --> ProductService : true / false
alt sesión activa
    ProductService -> ProductRepository : findAllActive()
    ProductRepository --> ProductService : List<Product>
    ProductService --> ProductController : OperationResult.ok(lista)
    ProductController --> InventoryView : showProducts(lista)
else sesión inactiva
    ProductService --> ProductController : OperationResult.error
    ProductController --> InventoryView : mostrar acceso no permitido
end
@enduml

```



4.5 RQF-013 - Edición de productos

El sistema carga una copia de los datos actuales. Las modificaciones solo se aplican cuando el administrador confirma Guardar.

Responsabilidades de implementación

- Buscar el producto por identificador.
- Cargar sus datos en ProductEditView.

- Conservar los valores originales mientras se edita.
- Validar los nuevos datos y la unicidad del SKU.
- Actualizar la entidad mediante el repositorio.
- Descartar la copia temporal cuando se selecciona Cancelar.

Diagrama de secuencia PlantUML - RQF-013

```
@startuml
actor Administrador
boundary ProductEditView
control ProductController
control ProductService
control ProductValidator
database ProductRepository

Administrador -> ProductEditView : selecciona Editar
ProductEditView -> ProductController : getProduct(id)
ProductController -> ProductService : getProductById(id)
ProductService -> ProductRepository : findById(id)
ProductRepository --> ProductService : Product
ProductService --> ProductController : producto
ProductController --> ProductEditView : loadProduct(product)

Administrador -> ProductEditView : modifica y guarda
ProductEditView -> ProductController : updateProduct(id, data)
ProductController -> ProductService : updateProduct(id, data)
ProductService -> ProductValidator : validar datos y SKU
ProductValidator --> ProductService : resultado
alt datos válidos
    ProductService -> ProductRepository : update(product)
    ProductRepository --> ProductService : producto actualizado
    ProductService --> ProductController : OperationResult.ok
    ProductController --> ProductEditView : mostrar confirmación
else datos inválidos
    ProductService --> ProductController : OperationResult.error
    ProductController --> ProductEditView : mostrar errores
end
@enduml
```

4.6 RQF-023 - Baja lógica de productos

La baja modifica el estado del producto a INACTIVE. No se elimina la entidad, lo que permite conservar su referencia en ventas anteriores.

Responsabilidades de implementación

- Seleccionar la opción Dar de baja.
- Mostrar ConfirmDeactivateDialog.
- Mantener el producto sin cambios cuando se cancela.
- Buscar el producto y verificar que esté activo.
- Cambiar su estado a INACTIVE.
- Excluirlo del inventario principal y de nuevas ventas.
- Mostrar el mensaje de operación exitosa.

Diagrama de secuencia PlantUML - RQF-023

```
@startuml
actor Administrador
boundary InventoryView
boundary ConfirmDeactivateDialog
control ProductController
control ProductService
database ProductRepository
```

```

Administrador -> InventoryView : selecciona Dar de baja
InventoryView -> ConfirmDeactivateDialog : abrir(productId)
ConfirmDeactivateDialog --> Administrador : Confirmar / Cancelar

alt Confirmar
  ConfirmDeactivateDialog -> ProductController : deactivateProduct(productId)
  ProductController -> ProductService : deactivateProduct(productId)
  ProductService -> ProductRepository : findById(productId)
  ProductRepository --> ProductService : Product
  ProductService -> ProductRepository : deactivate(productId)
  ProductRepository --> ProductService : true
  ProductService --> ProductController : OperationResult.ok
  ProductController --> ConfirmDeactivateDialog : "Producto dado de baja exitosamente"
  ConfirmDeactivateDialog --> InventoryView : actualizar lista
else Cancelar
  ConfirmDeactivateDialog --> InventoryView : cerrar sin cambios
end
@enduml

```

PARTE 5. TRAZABILIDAD ENTRE REQUISITOS Y COMPONENTES

ID	Vista	Lógica	Datos	Validaciones / resultado
RQF-007	ProductFormView	ProductController, ProductService, ProductValidator	ProductRepository, Product	Campos obligatorios, valores válidos, SKU único, mensaje de registro.
RQF-011	InventoryView	ProductController, ProductService, SessionManager	ProductRepository, Product	Sesión activa, lista de productos activos, inventario vacío.
RQF-013	ProductEditView	ProductController, ProductService, ProductValidator	ProductRepository, Product	Datos válidos, SKU único, conservación de valores al cancelar.
RQF-023	InventoryView, ConfirmDeactivateDialog	ProductController, ProductService, SessionManager	ProductRepository, Product	Confirmación, cambio a INACTIVE, exclusión de nuevas ventas.
RQF-001	LoginView	AuthController, AuthService, SessionManager	UserRepository, User	Campos completos, credenciales correctas, acceso permitido y sesión activa en SessionManager.
RQF-002	LoginView	AuthController, AuthService, SessionManager	UserRepository, User	Contraseña oculta, campos limpiados y credenciales no expuestas en mensajes.

5.1 Matriz de métodos mínimos

Clase	Métodos mínimos	Uso
ProductController	createProduct, listProducts, searchProducts, updateProduct, deactivateProduct	Punto de entrada desde Presentación.

Clase	Métodos mínimos	Uso
ProductService	registerProduct, getActiveProducts, searchProducts, updateProduct, deactivateProduct	Reglas del CRUD.
ProductValidator	validateRequiredFields, validateNumericValues, validateSkuUnique	Validaciones reutilizables.
ProductRepository	save, findAllActive, findById, findBySku, search, update, deactivate	Administración de registros.
SessionManager	getInstance, login, logout, isAuthenticated, getCurrentUser	Sesión única.
OperationResult	ok, error, getMessage, getData	Respuesta controlada hacia las vistas.

5.2 Validaciones y restricciones

Regla	Ubicación	Descripción
Campos obligatorios	ProductValidator.validateRequiredFields	Nombre, SKU, precio, cantidad y categoría deben contener datos.
Precio válido	ProductValidator.validateNumericValues	El precio debe ser numérico y mayor que cero.
Stock válido	ProductValidator.validateNumericValues	La cantidad debe ser entera y no negativa.
SKU único	ProductRepository.findBySku + ProductValidator	No se registra otro producto con el mismo SKU. En edición se excluye el producto actual.
Baja lógica	ProductService.deactivateProduct	El producto cambia a INACTIVE y no se elimina físicamente.
Sesión activa	SessionManager.isAuthenticated	Las operaciones protegidas requieren un administrador autenticado.
Cancelación de edición	ProductEditView	Los cambios temporales se descartan sin modificar el repositorio.
Confirmación de baja	ConfirmDeactivateDialog	La baja solo se ejecuta cuando el administrador confirma.
AuthController	login, logout	Punto de entrada desde LoginView.
AuthService	authenticate, closeSession	Validación de credenciales y coordinación de SessionManager.
UserRepository	findByEmail, validateCredentials	Administración y consulta de usuarios registrados.
Correo y contraseña obligatorios	AuthService.authenticate	No se procesa el acceso cuando uno de los campos está vacío.
Contraseña oculta	LoginView	Los caracteres ingresados no se muestran directamente en la pantalla.
Credenciales válidas	UserRepository.validateCredentials	El acceso se permite únicamente cuando correo y contraseña coinciden con un usuario registrado.
Limpieza de acceso	LoginView	Los campos se limpian después de un

Regla	Ubicación	Descripción
		intento incorrecto o al cerrar la sesión.

PARTE 6. ESTRUCTURA PROPUESTA DEL PROYECTO

Distribución de paquetes / carpetas

```

lib/
├── presentation/
│   ├── views/
│   │   ├── login_view.dart
│   │   ├── product_form_view.dart
│   │   ├── inventory_view.dart
│   │   └── product_edit_view.dart
│   ├── dialogs/
│   │   └── confirm_deactivate_dialog.dart
│   └── controllers/
│       ├── auth_controller.dart
│       └── product_controller.dart
├── business/
│   ├── services/
│   │   ├── auth_service.dart
│   │   └── product_service.dart
│   ├── validators/
│   │   └── product_validator.dart
│   ├── session/
│   │   └── session_manager.dart
│   └── dto/
│       ├── operation_result.dart
│       ├── product_dto.dart
│       └── search_criteria.dart
├── data/
│   ├── repositories/
│   │   ├── user_repository.dart
│   │   └── product_repository.dart
│   └── models/
│       ├── user.dart
│       ├── product.dart
│       ├── category.dart
│       └── product_status.dart
└── main.dart

```

La ubicación de controllers dentro del bloque presentation es una decisión física posible para una aplicación móvil, pero conceptualmente permanecen en el límite de coordinación de la Lógica de Negocio. Para mantener la lectura académica del modelo, los diagramas los clasifican dentro de Lógica de Negocio junto con los servicios.

PARTE 7. GUÍA DE CONSTRUCCIÓN VINCULADA A LOS SPRINTS

Sprint	Objetivo técnico	Componentes principales	Resultado verificable
Sprint 1	Registro y consulta inicial	Product, ProductRepository, ProductValidator, ProductService, ProductController, ProductFormView, InventoryView	Registrar productos válidos y visualizarlos en el inventario.
Sprint 2	Mantenimiento del inventario	SearchCriteria, ProductEditView, ConfirmDeactivateDialog, métodos search/update/deactivate	Buscar, editar, cancelar y dar de baja productos.

Sprint	Objetivo técnico	Componentes principales	Resultado verificable
Sprint 3	Control de acceso y ventas	User, UserRepository, AuthService, AuthController, SessionManager, módulo de venta	Activar una sesión única y proteger operaciones.
Sprint 4	Historial y alertas	Servicios y repositorios de ventas/alertas integrados a las mismas capas	Consultar historial y detectar stock mínimo.
Sprint 5	Escaneo y respaldo	Componentes de lectura, actualización y recuperación dentro de las capas existentes	Registrar por código y recuperar información administrada por la aplicación.

7.1 Orden recomendado para Sprint 1

6. Crear Product y ProductStatus.
7. Implementar ProductRepository con operaciones save, findAllActive, findBySku y findById.
8. Construir ProductValidator.
9. Implementar ProductService.registerProduct y getActiveProducts.
10. Crear ProductController como punto de entrada.
11. Construir ProductFormView e InventoryView.
12. Conectar el flujo de registro y actualización de la lista.
13. Probar campos vacíos, SKU repetido, valores inválidos e inventario vacío.

7.2 Orden recomendado para Sprint 2

14. Implementar SearchCriteria y ProductRepository.search.
15. Agregar búsqueda por nombre o SKU, filtros y ordenamiento.
16. Implementar ProductEditView con copia temporal de datos.
17. Crear ProductService.updateProduct y validar SKU durante edición.
18. Implementar ConfirmDeactivateDialog.
19. Crear ProductService.deactivateProduct y ProductRepository.deactivate.
20. Actualizar InventoryView para excluir productos INACTIVE.
21. Probar cancelación, confirmación y conservación del historial lógico.

7.3 Orden recomendado para Sprint 3

- Crear la entidad User y registrar las credenciales utilizadas por la aplicación.
- Implementar UserRepository con findByEmail y validateCredentials.
- Construir SessionManager con constructor privado, instancia estática y getInstance().
- Implementar AuthService.authenticate y AuthService.closeSession.
- Crear AuthController como punto de entrada desde LoginView.
- Construir LoginView con correo, contraseña oculta y mensajes de resultado.
- Conectar las operaciones protegidas de ProductService con SessionManager.isAuthenticated().
- Probar campos vacíos, credenciales incorrectas, acceso correcto y cierre de sesión.

PARTE 8. CRITERIOS DE ACEPTACIÓN ARQUITECTÓNICOS

Criterio	Aceptado cuando
Separación de capas	Ninguna vista importa o instancia directamente ProductRepository.
Coordinación	Las acciones de acceso entran por AuthController y las acciones CRUD por ProductController.

Criterio	Aceptado cuando
Reglas centralizadas	Las validaciones y cambios de estado se encuentran en Lógica de Negocio.
Persistencia encapsulada	Solo ProductRepository administra la colección de productos.
Singleton correcto	SessionManager tiene constructor privado, referencia estática y getInstance.
Singleton limitado	No existen ProductService, ProductRepository o controladores implementados como Singleton.
Baja lógica	El producto permanece registrado con estado INACTIVE.
Respuesta controlada	Las vistas reciben OperationResult con éxito, mensaje y datos.
Trazabilidad	Cada requisito de nivel 0 tiene al menos una vista, un componente lógico, un componente de datos y un criterio de prueba.

PARTE 9. MODELO ARQUITECTÓNICO GLOBAL

Código PlantUML del modelo global

```
@startuml
left to right direction
skinparam DefaultFontName Arial
skinparam shadowing false
skinparam roundcorner 10
skinparam componentStyle rectangle
skinparam packageStyle rectangle
skinparam defaultTextAlignment center

title PaperBoost - Modelo Arquitectónico Global\nTres Capas + Singleton + Requisitos Nivel 0

actor "Administrador" as Admin

package "Capa de Presentación\nInteracción con el usuario" #DDEBFA {
    component "LoginView" as Login
    component "ProductFormView" as Form
    component "InventoryView" as Inventory
    component "ProductEditView" as Edit
    component "ConfirmDeactivateDialog" as Dialog
}

package "Capa de Lógica de Negocio\nReglas, coordinación y patrón" #E6FAF0 {
    component "AuthController" as AuthC
    component "ProductController" as ProductC
    component "AuthService" as AuthS
    component "ProductService" as ProductS
    component "ProductValidator" as Validator
    component "OperationResult" as Result
    component "SessionManager\n<<Singleton>>" as Session
}

package "Capa de Datos\nRegistros y entidades" #FCE4D6 {
    component "UserRepository" as UserRepo
    component "ProductRepository" as ProductRepo
    component "User" as User
    component "Product" as Product
}
```

```

    component "ProductStatus" as Status
}

Admin --> Login
Admin --> Form
Admin --> Inventory
Admin --> Edit
Admin --> Dialog

Login --> AuthC : RQF-001 / RQF-002
AuthC --> AuthS
AuthS --> UserRepo
AuthS --> Session : activa sesión única
UserRepo o-- User
Session --> User : currentUser

Form --> ProductC : RQF-007 registrar
Inventory --> ProductC : RQF-011 consultar
Edit --> ProductC : RQF-013 actualizar
Dialog --> ProductC : RQF-023 baja lógica

ProductC --> ProductS
ProductC --> Result
ProductS --> Session : verifica acceso
ProductS --> Validator : valida campos / SKU
ProductS --> ProductRepo : CRUD
ProductS --> Result
ProductRepo o-- "0..*" Product
Product --> Status

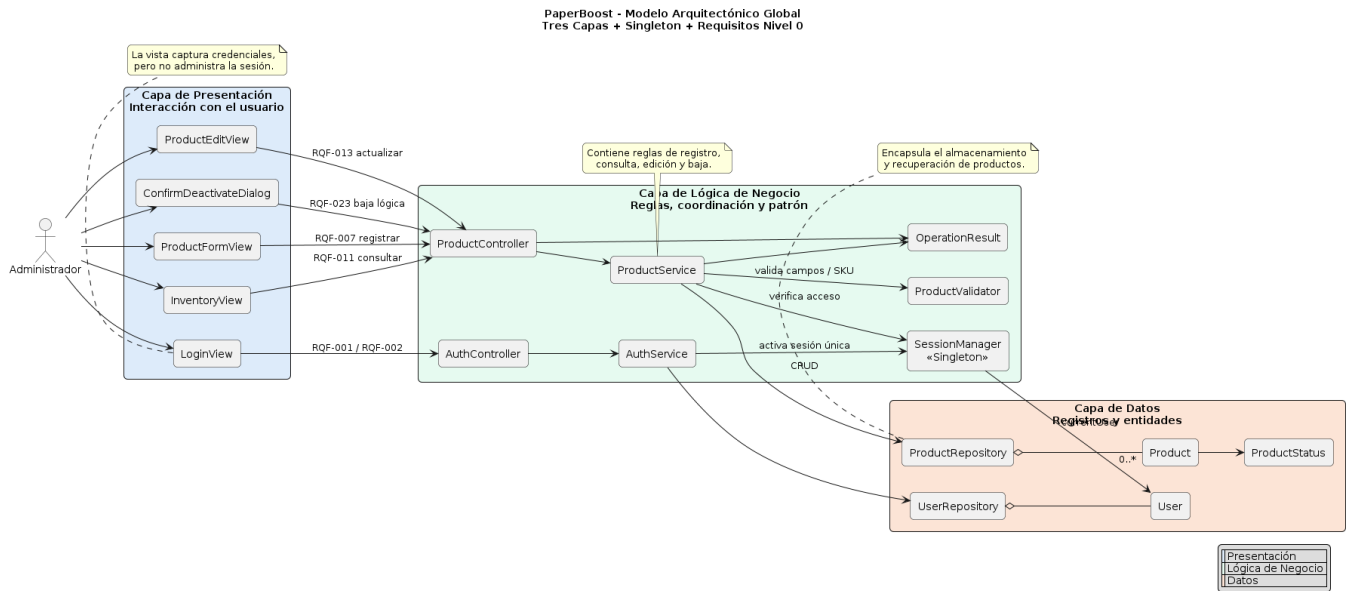
note bottom of Login
La vista captura credenciales,
pero no administra la sesión.
end note

note bottom of ProductS
Contiene reglas de registro,
consulta, edición y baja.
end note

note bottom of ProductRepo
Encapsula el almacenamiento
y recuperación de productos.
end note

legend right
|<#DDEBFA> | Presentación |
|<#E6FAF0> | Lógica de Negocio |
|<#FCE4D6> | Datos |
endlegend
@enduml

```



CONCLUSIONES

- La arquitectura de tres capas constituye la estructura principal de PaperBoost. Singleton se integra dentro de la capa lógica y no reemplaza la arquitectura.
- Los requisitos de nivel 0 se distribuyen entre vistas, controladores, servicios, validadores, repositorios y entidades, evitando que la interfaz concentre reglas, sesión o almacenamiento.
- SessionManager resuelve exclusivamente la necesidad de compartir una sesión activa para RQF-001 y RQF-002. La validación de credenciales permanece en AuthService y UserRepository, mientras la validación de SKU, edición, búsqueda y baja lógica permanecen en los componentes del módulo de productos.
- Los modelos PlantUML proporcionan una guía suficiente para crear paquetes, clases, dependencias, métodos y secuencias de ejecución.
- La baja lógica protege la consistencia del historial al conservar el producto con estado INACTIVE.
- La matriz de trazabilidad permite verificar que los seis requisitos principales de nivel 0 poseen componentes y criterios de aceptación identificables.

RECOMENDACIONES

- Mantener los nombres de clases y métodos consistentes entre PlantUML, código y Jira.
- Evitar accesos directos desde las vistas hacia los repositorios.
- Construir primero entidades y repositorios; posteriormente servicios, controladores y vistas dentro del mismo sprint.
- Realizar pruebas de ProductValidator y ProductService antes de conectar todas las pantallas.
- Restablecer SessionManager entre pruebas para evitar que una sesión afecte otros casos.
- Conservar ProductRepository como componente normal y no convertirlo en Singleton.
- Mantener actualizada la matriz de trazabilidad conforme se incorporen los demás requisitos planificados en los sprints.

REFERENCIAS DEL PROYECTO

- PaperBoost - Especificación de Requisitos de Software del equipo SolvEagle.
- PaperBoost - Análisis y Diseño de Software: Arquitectura de tres capas y patrón Singleton.

- G1_30723_U2T123 - Arquitectura de tres capas e integración de patrones de diseño.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Sommerville, I. Ingeniería de Software.